

Análise técnica do Cash4You

Pontos fortes, pontos fracos, segurança, usabilidade, critérios de mercado e plano de melhorias

Repositório	github.com/JoaoOliver/cash4you
Branches analisadas	main (base inicial) e devkbr (versão completa)
Stack	Python / Django 5.2, SQLite, Bootstrap 5.3, Chart.js
Tipo de análise	Revisão estática do código-fonte (arquivos enviados em ZIP)
Data	05/10/2026

Importante sobre o método. A análise é baseada na leitura do código. A aplicação não foi executada e os 126 testes automatizados não foram rodados (o ambiente de análise não tinha Django instalado nem acesso à internet). Afirmações sobre aparência e experiência de uso vêm da leitura dos templates, não de navegação real. Onde algo não pôde ser confirmado, o texto diz isso.

1. Resumo executivo

O Cash4You é um sistema web de gestão para pequenos negócios: clientes, produtos, serviços, frente de caixa (vendas), agenda, contas a pagar, precificação e relatórios. A branch **devkbr** é uma aplicação funcional com sete apps Django, regras de negócio bem pensadas (estoque atômico, precificação por markup, validação de CPF, agenda em blocos de 30 minutos) e boa cobertura de testes por quantidade. A branch **main** é uma base inicial, com usuário customizado, dashboard simples e modelos ainda sem telas completas.

O ponto mais crítico encontrado: nenhuma tela de negócio exige login. Não existe rota de login fora do `/admin/`, nenhuma view usa `login_required` ou verificação de permissão, e os grupos Administrador, Gerente e Funcionário criados pelo comando `criar_grupos` não são aplicados fora do painel admin. Na prática, quem alcançar a URL do sistema pode ver CPFs e telefones de clientes, registrar e excluir vendas e apagar cadastros.

Avaliação por critério

Critério	Situação	Comentário
Funcionalidade e regras de negócio	Forte	Escopo amplo e coerente para microempreendedores
Qualidade e organização do código	Média	Boa separação por apps; módulo core concentra lógica demais
Segurança	Fraca	Sem autenticação nas telas; sem hardening de produção
Privacidade / LGPD	Fraca	CPF e telefone em texto puro, sem controles de privacidade
Testes	Média	126 funções de teste, mas sem CI, sem medição de cobertura e app usuario sem testes
Usabilidade	Média a boa	Bootstrap, busca, filtros, confirmação de exclusão; não validada visualmente
Documentação	Forte	README extenso, docs de banco de dados, fluxos e diagramas SVG
Prontidão para produção	Fraca	SQLite, sem deploy, sem logs, sem CI/CD

2. Visão geral das branches

Aspecto	main	devkbr
Apps Django	catalogo, cliente, dashboard, usuario, vendaitem	produto, servico, cliente, usuario, agenda, venda, conta_pagar (mais o pacote core)
Telas	Listagens simples de produtos, serviços e clientes; dashboard com contagens	CRUD completo, caixa, histórico de vendas, agenda, contas a pagar, dashboard com calendário e gráficos, relatórios de caixa e estoque
Testes	Arquivos tests.py vazios (stubs padrão do Django)	126 funções de teste em 7 arquivos; o app usuario não tem nenhum
Documentação	README de uma linha; arquivo de controle semanal da disciplina	README detalhado, pasta docs com modelo de dados, fluxos e diagramas
Configuração	SECRET_KEY e DEBUG via variáveis de ambiente (python-decouple)	Igual

A **main** reflete o início do projeto, na fase de modelagem. Todo o restante da análise foca na **devkbr**, que representa o estado atual da aplicação. Os pontos que valem para as duas branches estão indicados.

Arquitetura

Monolito Django com templates server-side. Cada domínio fica em seu app, com models, forms, views e templates próprios. O pacote *core* concentra as telas transversais (dashboard, cadastros, relatórios), o módulo de precificação e o roteamento. A autenticação usa um modelo *Usuario* customizado, com login por e-mail e gerenciador próprio. O carrinho do caixa vive na sessão; a finalização grava venda, pagamentos e itens dentro de uma transação.

3. Pontos fortes

- **Integridade do estoque.** A finalização da venda roda em *transaction.atomic* e baixa o estoque com um UPDATE condicional (*quantidade__gte* com expressão *F()*). Isso evita estoque negativo e condições de corrida entre duas vendas simultâneas. A edição e a exclusão de vendas devolvem o estoque corretamente.
- **Histórico de vendas preservado.** O *ItemVenda* guarda descrição, preço unitário e subtotal no momento da venda, então mudar o preço de um produto depois não altera vendas antigas. Os dados do estabelecimento também são copiados para cada venda, o que mantém comprovantes antigos fiéis.
- **Dinheiro tratado com Decimal.** Valores monetários usam *DecimalField* e arredondamento *ROUND_HALF_UP*. O cálculo de preço por markup (*core/pricing.py*) protege contra divisão inválida quando despesas somadas ao lucro chegam a 100% ou mais.
- **Validações de domínio.** O CPF é validado pelos dígitos verificadores e rejeita sequências repetidas; o modelo *Cliente* chama *full_clean()* no *save()*, então a regra vale também fora dos formulários. A agenda tem restrição única por responsável, data e horário, e valida blocos de exatamente 30 minutos.
- **Boas práticas Django básicas.** Segredos e DEBUG vêm de variáveis de ambiente, com DEBUG falso por padrão; o *.gitignore* exclui *.env* e o banco; exclusões usam POST com tela de confirmação; o middleware CSRF está ativo; os validadores de senha do Django estão configurados; fuso e idioma estão corretos para o Brasil.
- **Recursos pensados para o público-alvo.** Pagamento múltiplo com troco, comprovante por WhatsApp (a mensagem é codificada com *quote()*), alerta de estoque baixo, limite de gastos mensal, calendário com feriados nacionais calculados (incluindo Páscoa e Carnaval).
- **Documentação acima da média** para um projeto acadêmico: README com requisitos, documentação do banco, fluxos e diagramas do modelo relacional.
- **Testes presentes em todos os módulos de negócio** (vendas 24, produtos 19, agenda 18, serviços 16, clientes 13, core 29, contas a pagar 7). Isso mostra preocupação com regressão, embora eu não tenha conseguido executá-los.

4. Pontos fracos

4.1 Falhas funcionais e de consistência de dados

- **Pagamento dividido distorce os relatórios.** A venda grava em *Venda.forma_pagamento* apenas a forma do primeiro pagamento. O dashboard e o relatório de caixa agrupam por esse campo, então numa venda paga metade em Pix e metade em dinheiro, o total inteiro é atribuído à primeira forma. O dado correto está na tabela *PagamentoVenda*, que os relatórios não usam.
- **Editar venda não atualiza os pagamentos.** A view *editar_venda* recalcula total, valor recebido e troco, mas não toca nos registros de *PagamentoVenda*. A tela de detalhe prefere exibir esses registros, então pode mostrar pagamentos que não batem com o novo total.
- **Limite de estoque baixo inconsistente.** O dashboard considera baixo o estoque abaixo de 5 unidades; o relatório de estoque considera baixo até 5 inclusive. O mesmo produto pode ser classificado de formas diferentes em duas telas.
- **Validação só na camada de formulário em Produto.** Os formulários de produto e serviço já impõem valor mínimo 0, mas no modelo *quantidade* é um *IntegerField* sem mínimo e os preços não têm validador. Cadastros feitos pelo admin, por scripts ou por futuras APIs não passam por essa regra. O ideal é repetir a validação no modelo.
- **Cliente sem unicidade e sem vínculo.** O CPF não é único (é possível cadastrar o mesmo CPF várias vezes). Vendas e agendamentos guardam nome e telefone como texto, sem ligação com o cadastro de clientes, então não há histórico de compras por cliente.
- **Sem rastreabilidade de quem fez o quê.** Venda e exclusões não registram o usuário responsável, e *excluir_venda* apaga o registro financeiro definitivamente. Para um sistema de caixa, isso dificulta auditoria e conferência.

4.2 Qualidade e manutenção do código

- **Módulo core sobrecarregado.** A view *dashboard* tem cerca de 200 linhas e mistura cálculo de calendário, feriados, consultas e montagem de contexto. Isso é difícil de testar isoladamente e de evoluir.
- **JavaScript e CSS inline em templates grandes.** *base.html* (cerca de 26 KB), *dashboard.html* (24 KB) e *caixa.html* (33 KB) concentram estilo e lógica no HTML. Isso impede uma política de segurança de conteúdo (CSP) restritiva e dificulta reaproveitamento.
- **Resíduos no repositório.** Existe a pasta *cliente/templates/olds* com templates antigos e o arquivo *docs/Documentacao_old.md*, que tem o mesmo tamanho do README (parece uma cópia). Em *devkbr*, os templates ficam soltos em pastas de app sem namespace (por exemplo *listar_cliente.html* e *listar_produto.html*), o que pode gerar colisão de nomes.
- **Desempenho em volume.** As listagens de vendas, produtos e clientes não têm paginação; o dashboard carrega as vendas do período em memória e soma em Python. Funciona bem para pequenos volumes, mas degrada com o crescimento.
- **Arquivo requirements.txt em UTF-16.** O pip costuma ler, mas outras ferramentas (Docker, Dependabot, scanners de dependência) podem falhar. O ideal é UTF-8. Também não há separação de dependências de desenvolvimento e produção.

5. Segurança

A tabela resume os achados, ordenados por gravidade. Todos foram confirmados por leitura do código, exceto onde indicado.

#	Achado	Gravidade	Evidência	Recomendação
S1	Nenhuma tela exige autenticação ou permissão	Crítica	Busca por login_required, PermissionRequired e LOGIN_URL não retorna nada; core/urls.py só tem login via /admin/	Exigir login em todas as views (mixin ou middleware); criar telas de login e logout; definir LOGIN_URL
S2	Grupos e permissões não são aplicados	Alta	criar_grupos define permissões só para usuario, produto, servico e cliente; venda, agenda e conta_pagar ficam fora; as views não consultam permissões	Aplicar permission_required por view; incluir os apps novos nos grupos; testar cada perfil
S3	CPF e telefone em texto puro, sem controles de LGPD	Alta	Cliente.cpf como CharField; sem política de privacidade, consentimento ou rotina de exclusão	Minimizar dados, criptografar o CPF em repouso, registrar base legal e prever exclusão e anonimização
S4	Sem hardening de produção	Alta	settings.py não define SECURE_SSL_REDIRECT, HSTS, SESSION_COOKIE_SECURE, CSRF_COOKIE_SECURE, CSRF_TRUSTED_ORIGINS nem logging	Ativar as opções SECURE_* quando DEBUG for falso; rodar manage.py check --deploy
S5	Painel /admin/ no caminho padrão e sem proteção contra força bruta	Média	core/urls.py expõe admin/ sem limitador de tentativas	Trocar o caminho, usar django-axes ou similar e considerar 2FA para administradores
S6	Scripts e estilos de CDN sem Subresource Integrity	Média	Bootstrap CSS e Chart.js carregados do jsDelivr sem atributo integrity; o Chart.js sem versão fixada	Fixar versões, adicionar integrity e crossorigin, ou servir os arquivos localmente
S7	Exclusão definitiva de registros financeiros sem trilha	Média	excluir_venda executa venda.delete(); nenhum log de auditoria	Usar exclusão lógica (campo cancelada) e registrar usuário, data e motivo
S8	Valores digitados reinseridos via innerHTML	Baixa	caixa.html monta HTML com value="\$ {...}" a partir de campos já digitados, sem escape	Montar elementos com DOM API (createElement, value) em vez de concatenar HTML; severidade baixa porque o dado é do próprio usuário
S9	SQLite como banco único	Baixa	DATABASES usa sqlite3; select_for_update não tem efeito em SQLite	Migrar para PostgreSQL antes de colocar em produção com mais de um usuário

Pontos positivos de segurança: segredos fora do código, DEBUG desligado por padrão, CSRF ativo, uso do ORM (sem SQL cru, o que afasta injeção de SQL), exclusões somente por POST, validação de entradas de data e período nas consultas, e os dados do comprovante de WhatsApp codificados na URL.

6. Usabilidade e acessibilidade

Esta seção vem da leitura dos templates e das views. Não houve teste em navegador, então a avaliação visual e a responsividade real precisam ser confirmadas.

O que funciona bem

- Interface baseada em Bootstrap 5, que tende a ser responsiva por padrão.
- Busca e ordenação nas listagens de produtos e clientes (por nome, CPF, telefone, preço e quantidade); filtros por forma de pagamento e período no histórico de vendas.
- Mensagens de feedback (sucesso e erro) após ações, como "Item removido do carrinho" e "Estoque insuficiente para X. Disponível: N", que dizem ao usuário o que fazer.
- Confirmação antes de excluir, e dashboard com visão por dia, semana e mês, calendário com feriados e vencimentos de contas.
- Caixa com campos rotulados e atributos *aria-label* nos campos de pagamento dinâmicos.

O que melhorar

- **Sem fluxo de login e perfis visíveis.** Como não há autenticação, o sistema não mostra nome do usuário, perfil nem botão de sair.
- **Acessibilidade não verificada de ponta a ponta.** Convém testar contraste, navegação por teclado, foco visível e leitores de tela (WCAG 2.1 AA), principalmente no caixa e nos gráficos, que não têm alternativa textual evidente.
- **Gráficos dependem de JavaScript e CDN.** Sem internet o dashboard perde os gráficos. Uma tabela equivalente ajudaria acessibilidade e uso offline.
- **Listagens sem paginação** (não encontrei uso de Paginator no projeto) tornam a navegação lenta e longa quando os dados crescem.
- **Formulários:** máscara e validação em tempo real para CPF e telefone, e mensagens de erro junto ao campo, reduzem retrabalho. O caixa é o ponto de maior uso e merece atalhos de teclado e leitura de código de barras.
- **Mobile:** como o público é de microempreendedores, vale testar o caixa e a agenda em celular real e considerar PWA.

7. Critérios que o mercado exige hoje

Critério	Estado atual	Observação
Autenticação e controle de acesso (RBAC)	Ausente nas telas	Prioridade máxima; ver seção 5
LGPD (Lei 13.709/2018)	Ausente	Dados pessoais (CPF, telefone, e-mail) sem política, consentimento nem rotinas de direito do titular
Trilha de auditoria	Ausente	Sem registro de usuário em vendas, edições e exclusões
Testes automatizados	Parcial	126 funções; sem medição de cobertura; app usuario sem testes
Integração contínua (CI)	Não encontrada	Sem pasta .github nem pipeline; ideal rodar testes, lint e verificação de segurança a cada push
Containerização e deploy	Não encontrados	Sem Dockerfile, servidor WSGI/ASGI de produção nem arquivos estáticos configurados (sem STATIC_ROOT)
Banco de dados de produção	SQLite	Migrar para PostgreSQL; ter rotina de backup e restauração
Observabilidade	Ausente	Sem configuração de logging, monitoramento de erros ou métricas
API REST / integrações	Ausente	Apenas link do WhatsApp; falta API para app mobile e integrações (Pix, contabilidade)
Documento fiscal (NFC-e / NF-e / recibo)	Ausente	O comprovante por WhatsApp não é documento fiscal; relevante para negócios formalizados
Pix com QR Code	Ausente	Pix só é registrado como forma de pagamento, sem geração de cobrança
Multiempresa / multiusuário (SaaS)	Ausente	DadosEstabelecimento é um registro único (pk=1); sem separação de dados por empresa
Acessibilidade e responsividade	Parcial	Bootstrap ajuda; falta validação formal
Documentação	Boa	Manter atualizada e remover arquivos duplicados

8. Sugestões de melhoria

Organizadas por prioridade. Cada item indica o esforço aproximado para um desenvolvedor que já conhece o projeto.

Prioridade 1 — antes de qualquer uso real (1 a 3 dias)

- **Exigir autenticação em todo o sistema.** A forma mais rápida é um middleware que redireciona não autenticados, mantendo o login e os estáticos liberados. Alternativa mais explícita: *LoginRequiredMixin* ou *@login_required* em cada view.

```
# core/settings.py
LOGIN_URL = "login"
LOGIN_REDIRECT_URL = "home"

# core/urls.py
from django.contrib.auth import views as auth_views
path("login/", auth_views.LoginView.as_view(template_name="login.html"), name="login"),
path("logout/", auth_views.LogoutView.as_view(), name="logout"),

# Django 5.1+ tem LoginRequiredMixin (exige login por padrão):
MIDDLEWARE += ["django.contrib.auth.middleware.LoginRequiredMixin"]
# e nas views públicas (login) usar @login_not_required
```

- **Aplicar os perfis.** Usar *permission_required* nas views (por exemplo, só Gerente e Administrador excluem venda; Funcionário opera o caixa). Atualizar *criar_grupos* para incluir os apps venda, agenda e conta_pagar e escrever testes de acesso por perfil.
- **Endurecer a configuração de produção:**

```
# settings.py (quando DEBUG = False)
SECURE_SSL_REDIRECT = True
SECURE_HSTS_SECONDS = 31536000
SESSION_COOKIE_SECURE = True
CSRF_COOKIE_SECURE = True
SECURE_CONTENT_TYPE_NOSNIFF = True
# validar com: python manage.py check --deploy
```

Prioridade 2 — correteude dos dados (2 a 4 dias)

- **Corrigir os relatórios de pagamento** para somar por *PagamentoVenda* em vez de *Venda.forma_pagamento*:

```
from venda.models import PagamentoVenda
PagamentoVenda.objects.filter(venda__criada_em_gte=ini, venda__criada_em_lt=fim) \
    .values("forma_pagamento").annotate(total=Sum("valor")).order_by("-total")
```

- **Atualizar PagamentoVenda ao editar a venda** (apagar e recriar dentro da mesma transação) ou restringir a edição a campos que não afetam pagamento.
- **Unificar o limite de estoque baixo** numa constante (ou campo configurável) usada por dashboard e relatório.
- **Tornar o CPF único** (*unique=True*, com migração que trata duplicados existentes) e ligar *Venda* e *Agendamento* ao *Cliente* por chave estrangeira opcional.
- **Repetir no modelo as validações dos formulários:** *MinValueValidator(0)* em quantidade e preços de Produto e Serviço.
- **Registrar o responsável:** campo *usuario* em *Venda* e, de preferência, cancelamento lógico em vez de exclusão (campos *cancelada_em*, *cancelada_por*, *motivo*).

Prioridade 3 — qualidade e manutenção (1 a 2 semanas)

- Extrair a lógica do dashboard para módulos de serviço (*core/services/calendario.py*, *core/services/indicadores.py*) e testar cada um.
- Mover CSS e JavaScript inline para arquivos estáticos; isso abre caminho para CSP. Fixar versão e integridade do Chart.js ou servir localmente.

- Namespace nos templates (*produto/listar.html* em vez de *listar_produto.html* solto) e remoção de *olds* e *Documentacao_old.md*.
- Paginação (*Paginator*) nas listagens e agregações feitas no banco, usando *TruncDate* e *Sum* em vez de somar em Python.
- Converter *requirements.txt* para UTF-8 e separar *requirements-dev.txt* (ferramentas de teste e lint).

Prioridade 4 — preparar para o mercado (semanas)

- **CI no GitHub Actions** rodando testes, *ruff* (lint), *coverage* e *pip-audit* (vulnerabilidades em dependências) a cada push e pull request.
- **Deploy** com Dockerfile, gunicorn, WhiteNoise para estáticos, PostgreSQL e configuração por variáveis de ambiente; backups automáticos.
- **LGPD**: política de privacidade, registro de consentimento, exportação e exclusão de dados de um cliente, criptografia do CPF (por exemplo com *django-fernet-fields* ou similar) e retenção definida.
- **Observabilidade**: *LOGGING* estruturado e Sentry (ou equivalente) para erros.
- **Funcionalidades de mercado**: Pix com QR Code, emissão fiscal via API de terceiros, API REST (Django REST Framework) para um app mobile, exportação de relatórios em PDF e Excel, e modelo multiempresa se o objetivo for SaaS.
- **Experiência de uso**: PWA para o caixa em celular, atalhos de teclado e leitor de código de barras, máscaras de CPF e telefone, tabelas alternativas aos gráficos e revisão WCAG 2.1 AA.

9. Conclusão e próximos passos

O Cash4You já entrega um conjunto de funcionalidades sólido para o público de pequenos negócios, e o cuidado com estoque transacional, precificação e documentação mostra boa base de engenharia. O que separa o projeto de um uso real é principalmente a camada de **acesso e proteção de dados**: hoje o sistema funciona como se todos fossem administradores e não exige identificação.

Sugestão de ordem de trabalho: (1) autenticação e permissões; (2) correção dos relatórios de pagamento e da edição de venda; (3) validações e unicidade nos modelos; (4) CI, testes de acesso por perfil e hardening de produção; (5) LGPD, deploy e recursos de mercado.

Limitações desta análise

- O código não foi executado e os testes não foram rodados; falhas de execução ou testes quebrados podem existir sem aparecer aqui.
- Não li todos os arquivos linha a linha. A leitura se concentrou em configuração, modelos, views, comandos de gestão, requisitos e nos templates de maior risco; os arquivos de formulários, os arquivos de teste e os templates menores foram vistos por tamanho, nome e busca de padrões.
- Aparência, responsividade e acessibilidade precisam de teste em navegador e dispositivos reais.
- O repositório local não incluía histórico Git, então não analisei o histórico de commits além do que aparece nos pull requests públicos (usuário customizado, grupos, dashboard e vendaitem).